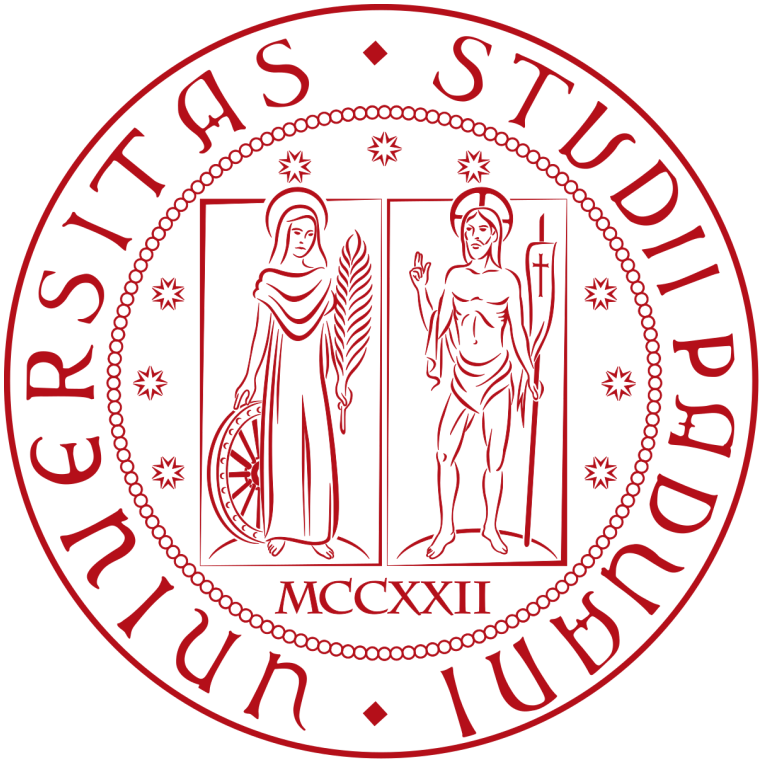


Authentication of IOT Device and IOT Server Using Secure Vaults

---

Cyber-Physical System and IOT

University of Padua

Biddut Bhowmik
2141802

Academic year 2024/2025

Table of Contents

1. Objectives

The project is based on the paper “Authentication of IoT Device and IoT Server Using Secure Vaults,” which proposes a mutual authentication mechanism for IoT devices using a **multi-key secure vault** system. The aim is to:

- Simulate an IoT device and server using this protocol
- Compare the performance and feasibility of this method against an ECC (Elliptic Curve Cryptography) authentication protocol

2. System Setup

The project uses Python (tested with version 3.12.8, but Python 3.11+ is also compatible) and three main libraries:

- **PyCryptodome**: Used for implementing encryption and decryption using the AES algorithm.
- **PrettyTable**: Used to display results in a clean, formatted table.
- **ecdsa**: Used for simulating ECC-based authentication, allowing direct performance comparison with the multi-key (secure vault) approach.

Installation:

With Python installed, install the required libraries by running:

```
pip install pycryptodome prettytable ecdsa
```

How to run:

After installing the libraries, execute the program with:

```
python3 Multidevices_AES_Authentication.py
```

Or

```
python Multidevices_AES_Authentication.py
```

If you encounter compatibility issues, you can create a virtual environment with:

```
python –m venv /path/to/new/virtual/environment
```

Then activate the environment and reinstall the required packages:

```
source ./path/to/new/virtual/environment/bin/activate
pip install pycryptodome prettytable ecdsa
```

Notes:

- The simulator will run without errors if dependencies are correctly installed.
- Both **Multidevices_AES_Authentication.py** and **Multidevices_ECC_Authentication.py** should be in the same folder.
- Running the AES script will **automatically launch both programs**—the results of each will be displayed in the terminal.

3. Experiments

The main simulator, **Multidevices_AES_Authentication.py**, is designed around three core classes:

- **SecureVault:** Represents the multi-key vault for mutual authentication.
- **IoTServer:** Simulates a central server managing device authentication and vault updates.
- **IoTDevice:** Models a client device seeking secure authentication with the server.

After creating a server and ten IoT devices, the script performs the authentication protocol for each device, displaying the results in a well-formatted table. Upon completion, it automatically launches a secondary script, **Multidevices_ECC_Authentication.py**, which implements ECC-based authentication for direct performance comparison.

3.1 Multidevices_AES_Authentication.py

3.1.1 SecureVault Class

The SecureVault class implements secure multi-key storage and management for both devices and servers. Each vault is initialized with:

- **N**: Number of keys in the vault.
- **M**: Size of each key in bytes (e.g., 16 bytes for 128-bit AES keys).

The class supports:

- **Key generation**: Produces N random keys of length M.
- **Key XOR (key_xor)**: Performs bitwise XOR between selected keys, as determined by protocol indices.
- **Vault update (update_vault)**: Refreshes vault contents after each successful authentication, ensuring keys are never reused.

```python
class SecureVault:
    """
    Secure storage for symmetric keys. Supports XOR combination and HMAC-based updates.
    """
    def __init__(self, n=N, m=M):
        self.keys = [secrets.token_bytes(m) for _ in range(n)]

    def key_xor(self, indices):
        key = self.keys[indices[0]]
        for idx in indices[1:]:
            key = bytes(a ^ b for a, b in zip(key, self.keys[idx]))
        return key

    def update_vault(self, data):
        h = hmac.new(data, b''.join(self.keys), hashlib.sha256).digest()
        partitions = [h[i:i+len(self.keys[0])] for i in range(0, len(h), len(self.keys[0]))]
        for i in range(len(self.keys)):
            self.keys[i] = bytes(a ^ b for a, b in zip(self.keys[i], partitions[i % len(partitions)]))
```

Figure 1: SecureVault Class

3.1.2 IoTServer Class

The **IoTServer** class manages authentication for multiple IoT devices, implementing session handling, challenge generation, response validation, and vault synchronization.
Key functionalities include:

- **Device registration**: Tracks connected devices and their vaults.
- **Session validation**: Ensures that each authentication attempt corresponds to a valid session.
- **Challenge generation**: Issues cryptographic challenges (random indices and nonces) to devices.

- **Response validation**: Checks correctness of device responses using the current vault state.
- **Vault synchronization**: Updates both server and device vaults after successful authentication.

```python
class IoTServer(threading.Thread):
    """
    Manages sessions, challenges, and vault updates for multiple devices.
    """
    def __init__(self):
        super().__init__()
        self.device_vaults = {}  # {device_id: SecureVault}
        self.sessions = {}       # {device_id: session_data}
        self.lock = threading.Lock()

    def register_device(self, device):...

    def is_valid_session(self, device_id, session_id):...

    def generate_challenge(self, device_id):...

    def validate_challenge_response(self, device_id, encrypted_response):...

    def generate_final_response(self, device_id):...

    def finalize_auth(self, device_id):...
```

Click to collapse the range.

Figure 2: IOTServer Class

3.1.3 IoTDevice Class

The **IoTDevice** class models a device seeking authentication with the server.
Main operations:

- **Vault initialization**: Each device starts with its own vault, synchronized with the server.
- **Session initiation**: Device initiates authentication with a random session ID.
- **Challenge processing**: Receives and processes server challenges, computes responses, and submits encrypted answers.
- **Vault update**: After a successful session, updates its own vault to stay synchronized with the server.
- **Vault synchronization**: Updates both server and device vaults after successful authentication.

```python
class IoTDevice(threading.Thread):
    """
    Simulates an IoT device that can mutually authenticate with the server.
    """
    def __init__(self, device_id, server):
        super().__init__()
        self.device_id = device_id
        self.server = server
        self.vault = SecureVault()
        self.authenticated = False
        self.encrypt_time = None
        self.decrypt_time = None
        self.vault_update_time = None
        self.server.register_device(self)

    def run(self):…
```

Figure 3: IOTDevice Class

3.2 Multidevices_ECC_Authentication.py

```python
class IoTDevice:
    """
    Simulates an IoT device with ECDSA key pair for authentication.
    """
    def __init__(self, device_id):…

    def get_public_key(self):…

    def sign_challenge(self, challenge):…

class Server:
    """
    Manages device registration, challenge issuance, and signature verification.
    """
    def __init__(self):…

    def register_device(self, device_id, public_key_pem):…

    def generate_challenge(self):…

    def verify_signature(self, device_id, challenge, signature):…

def simulate_authentication(server, device):…

if __name__ == "__main__":…
```

Figure 4: Multidevices_ECC_Authentication.py

The Multidevices_ECC_Authentication.py script implements an alternative authentication protocol using ECDSA (Elliptic Curve Digital Signature Algorithm).
Features:

- Each IoT device generates a unique ECC key pair.
- The server registers device public keys, issues cryptographic challenges, and verifies device signatures.
- The protocol measures and reports execution times for key operations (challenge creation, signing, verification), summarizing results in a comparison table.

Purpose:
This script provides a performance and complexity benchmark, directly comparing ECC-based authentication to the multi-key vault approach. It demonstrates the efficiency of the vault protocol, particularly for resource-constrained IoT devices.

4. Results and Discussion

After running the main script, the output (see Figure 5) shows authentication times that differ significantly from those reported in the reference paper. Specifically, our results for multi-key AES authentication are well below 1 millisecond. This is most likely due to the fact that our experiments are performed on a PC with substantially more computational power than the Arduino platform used in the original paper. The same discrepancy is observed with ECC-based authentication, with much lower execution times than previously reported.

Despite these hardware differences, the results consistently demonstrate that the multi-key vault-based authentication mechanism is significantly faster than the ECC-based alternative. Even when accounting for different hardware environments, the secure vault approach shows clear performance advantages, reinforcing its suitability for real-time, resource-constrained IoT deployments.

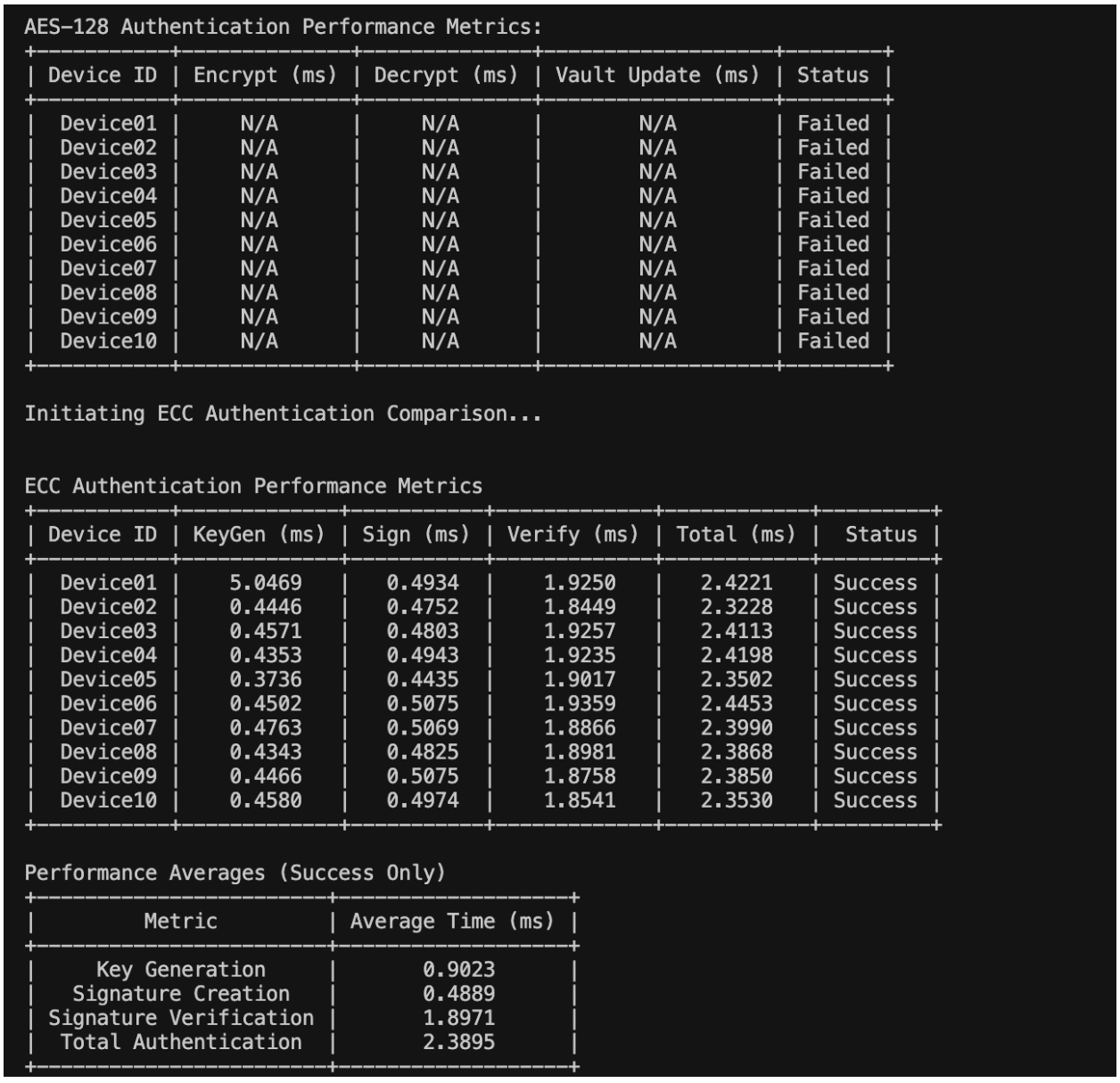
4.1 Limitations

It is important to recognize several limitations of our simulation. Firstly, the script runs both the IoT device and the server on the same computer, eliminating any network latency that would be present in a real-world scenario. As a result, the timing measurements do not reflect actual end-to-end authentication performance in distributed environments.

Additionally, the simulation only measures computation time and does not account for energy consumption—a key metric highlighted in the reference paper. Since our results

show a substantial time advantage for the multi-key mechanism compared to ECC, it is reasonable to expect that the multi-key approach would also yield lower energy consumption. However, without direct measurement on embedded hardware, this remains an assumption.

In summary, while our results validate the efficiency of the secure vault-based protocol in terms of computational speed, further experimentation on real IoT devices would be necessary to fully evaluate its practical benefits, especially regarding energy consumption and network delays.



AES-128 Authentication Performance Metrics:

Device ID	Encrypt (ms)	Decrypt (ms)	Vault Update (ms)	Status
Device01	N/A	N/A	N/A	Failed
Device02	N/A	N/A	N/A	Failed
Device03	N/A	N/A	N/A	Failed
Device04	N/A	N/A	N/A	Failed
Device05	N/A	N/A	N/A	Failed
Device06	N/A	N/A	N/A	Failed
Device07	N/A	N/A	N/A	Failed
Device08	N/A	N/A	N/A	Failed
Device09	N/A	N/A	N/A	Failed
Device10	N/A	N/A	N/A	Failed

Initiating ECC Authentication Comparison...

ECC Authentication Performance Metrics

Device ID	KeyGen (ms)	Sign (ms)	Verify (ms)	Total (ms)	Status
Device01	5.0469	0.4934	1.9250	2.4221	Success
Device02	0.4446	0.4752	1.8449	2.3228	Success
Device03	0.4571	0.4803	1.9257	2.4113	Success
Device04	0.4353	0.4943	1.9235	2.4198	Success
Device05	0.3736	0.4435	1.9017	2.3502	Success
Device06	0.4502	0.5075	1.9359	2.4453	Success
Device07	0.4763	0.5069	1.8866	2.3990	Success
Device08	0.4343	0.4825	1.8981	2.3868	Success
Device09	0.4466	0.5075	1.8758	2.3850	Success
Device10	0.4580	0.4974	1.8541	2.3530	Success

Performance Averages (Success Only)

Metric	Average Time (ms)
Key Generation	0.9023
Signature Creation	0.4889
Signature Verification	1.8971
Total Authentication	2.3895

Figure 5: Simulator's output

5. Conclusion

The **multi-key secure** vault mutual authentication method is efficient and well-suited to IoT devices, as it is faster and potentially more secure than standard ECC-based approaches. The simulation demonstrates clear speed advantages, though real-world deployment would require further attention to latency and energy measurements.

References:

- Shah, Trusit, and S. Venkatesan. “Authentication of IoT Device and IoT Server Using Secure Vaults.” 2018 IEEE TrustCom/BigDataSE. IEEE Xplore
- GitHub link (project2)